Mass-Storage Systems



Overview of Mass Storage Structure



- **Magnetic disks** provide bulk of secondary storage of modern computers
  - Drives rotate at 60 to 250 times per second
  - **Transfer rate** - rate of data flow between drive and computer
  - **Positioning time** (**random-access time**) = **seek time+ rotational latency**

    seek time = time to move disk arm to desired cylinder

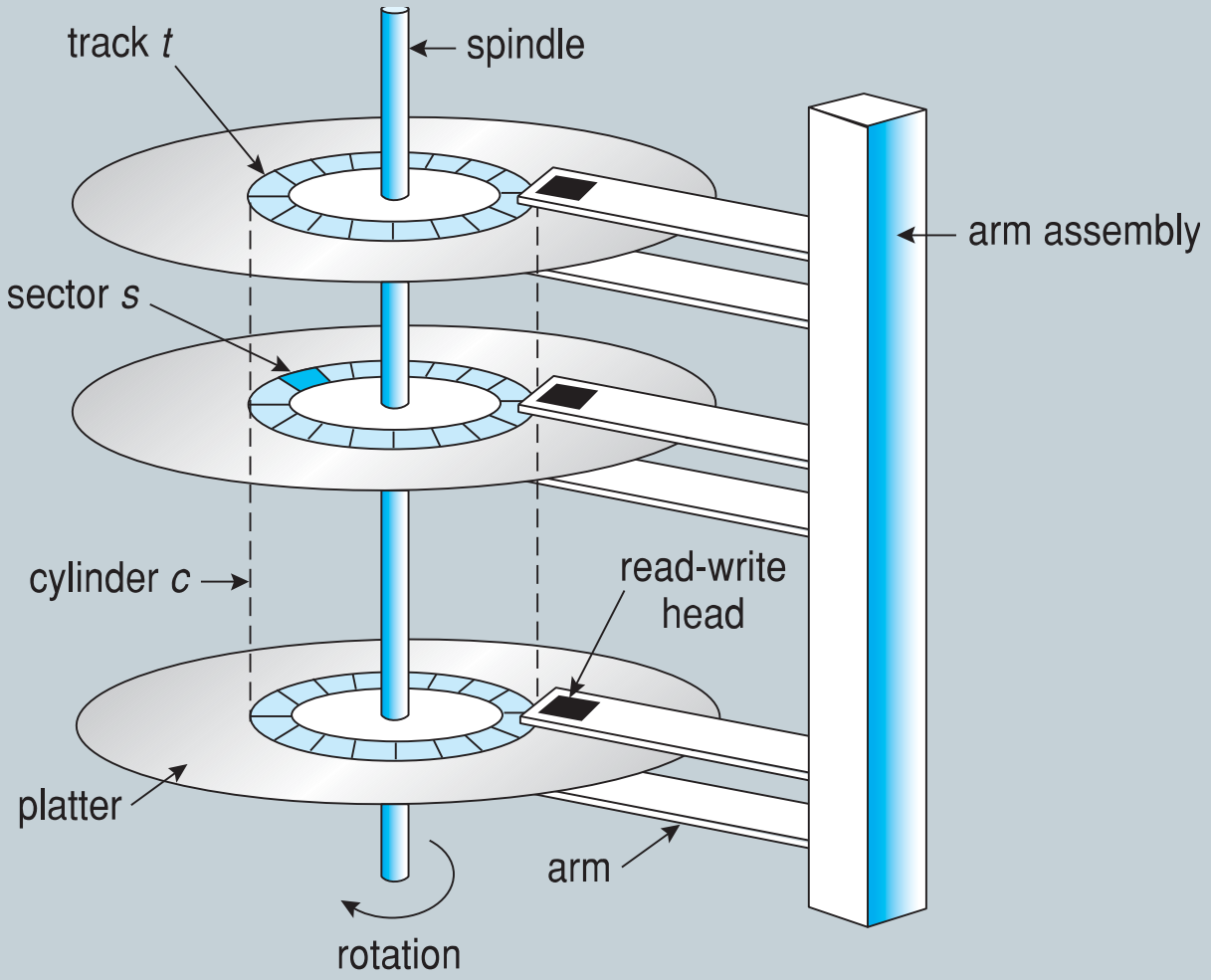
    rotational latency = time for desired sector to rotate under the disk head
  - **Head crash** results from disk head making contact with the disk surface

Overview of Mass Storage Structure cont...



- Disks can be removable
- Drive attached to computer via **I/O bus**
  - Busses vary, including **EIDE**, **ATA**, **SATA**, **USB**, **Fibre Channel**, **SCSI**, **SAS**, **Firewire**
  - **Host controller** in computer uses bus to talk to **disk controller** built into drive or storage array

Moving-head Disk Mechanism



Hard Disks Performance

- Performance
  - Transfer Rate – theoretical – 6 Gb/sec
  - Effective Transfer Rate – real – 1Gb/sec
  - Seek time from 3ms to 12ms – 9ms common for desktop drives
  - Average seek time measured or calculated based on 1/3 of tracks
  - Latency based on spindle speed
    - 1 / (RPM / 60) = 60 / RPM
  - Average latency = ½ latency

Spindle [rpm]	Average latency [ms]
4200	7.14
5400	5.56
7200	4.17
10000	3
15000	2

(From Wikipedia)

Hard Disks Performance

- **Access Latency** = **Average access time** = average seek time + average latency
  - For fastest disk 3ms + 2ms = 5ms
  - For slow disk 9ms + 5.56ms = 14.56ms
- Average I/O time = average access time + (amount to transfer / transfer rate) + controller overhead

Solid-State Disks



- More reliable than HDDs
- More expensive per MB
- Maybe have shorter life span
- Less capacity
- Faster (No moving parts, so no seek time or rotational latency)

Magnetic Tape

- Secondary-storage medium
- Relatively permanent and holds large quantities of data
- Access time slow (Random access ~1000 times slower than disk)
- Mainly used for backup, storage of infrequently-used data, transfer medium between systems

Disk Structure

- Disk drives are addressed as large 1-dimensional arrays of **logical blocks**,
- logical block
  - smallest unit of transfer
  - mapped into the sectors of the disk sequentially
    - Sector 0 is the first sector of the first track on the outermost cylinder

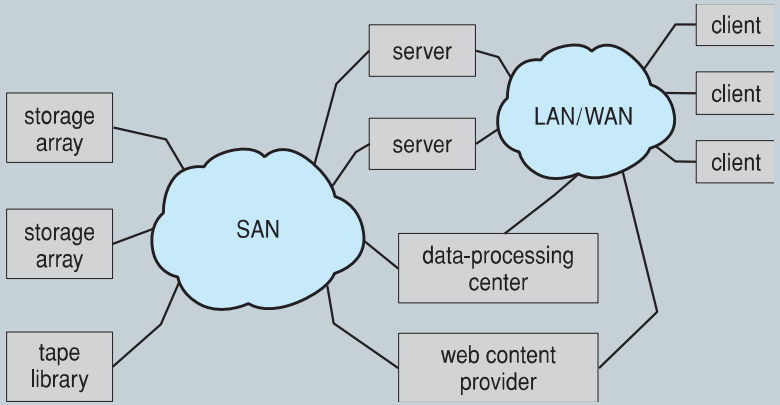
Storage Array



- Can just attach disks, or arrays of disks
- Storage Array has controller(s), provides features to attached host(s)

Storage Area Network

- large storage environments (one or more storage arrays) which connect through the speed network connection (Fibre Channel)
- Multiple hosts attached to multiple storage arrays – flexible
- Easy to add or remove storage, add new host and allocate it storage

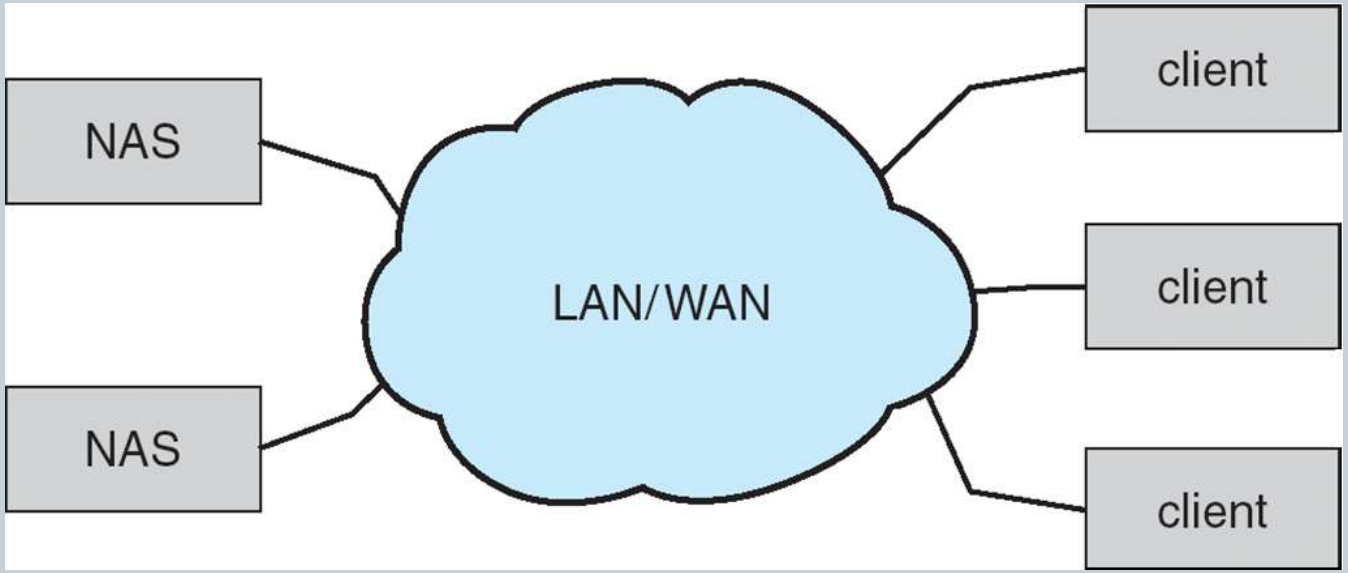


Network-Attached Storage



- Storage made available over a network rather than over a local connection (such as a bus)
  - Remotely attaching to file systems
- Implemented via remote procedure calls (RPCs) between host and storage over typically TCP or UDP on IP network
- **iSCSI** protocol uses IP network to carry the SCSI protocol
  - Remotely attaching to devices (blocks)

Network-Attached Storage



Disk Scheduling



- OS is responsible for using hardware efficiently Minimize seek time
- Seek time ≈ seek distance
- Disk **bandwidth** is the total number of bytes transferred per unit time

Disk Scheduling (Cont.)



- Sources of disk I/O request
  - OS
  - System processes
  - Users processes
- I/O request includes input or output mode, disk address, memory address, number of sectors to transfer
- OS maintains queue of requests (per disk or device)
- Idle disk can immediately work on I/O request, busy disk means work must queue

Disk Scheduling (Cont.)



- Drive controllers have small buffers and can manage a queue of I/O requests (of varying “depth”)
- Several algorithms exist to schedule the servicing of disk I/O requests

Disk Scheduling (Cont.)

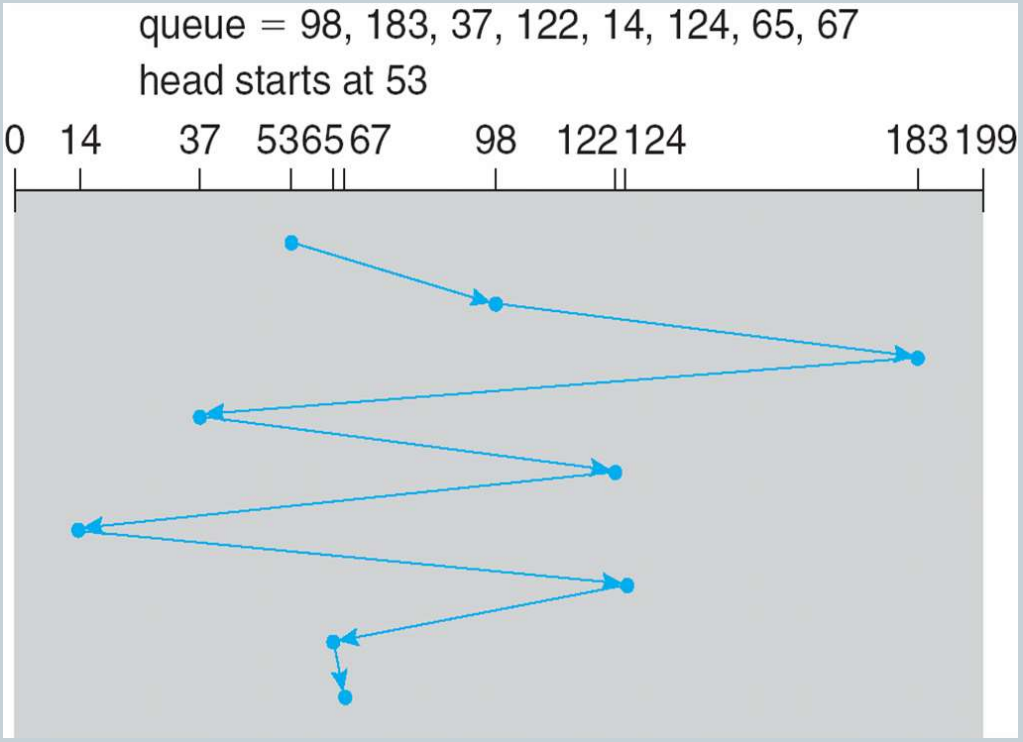
- illustrate scheduling algorithms with a request queue (0-199)

  98, 183, 37, 122, 14, 124, 65, 67

  Head pointer 53

First-Come, First-Served (FCFS)

- Illustration shows total head movement of 640 cylinders

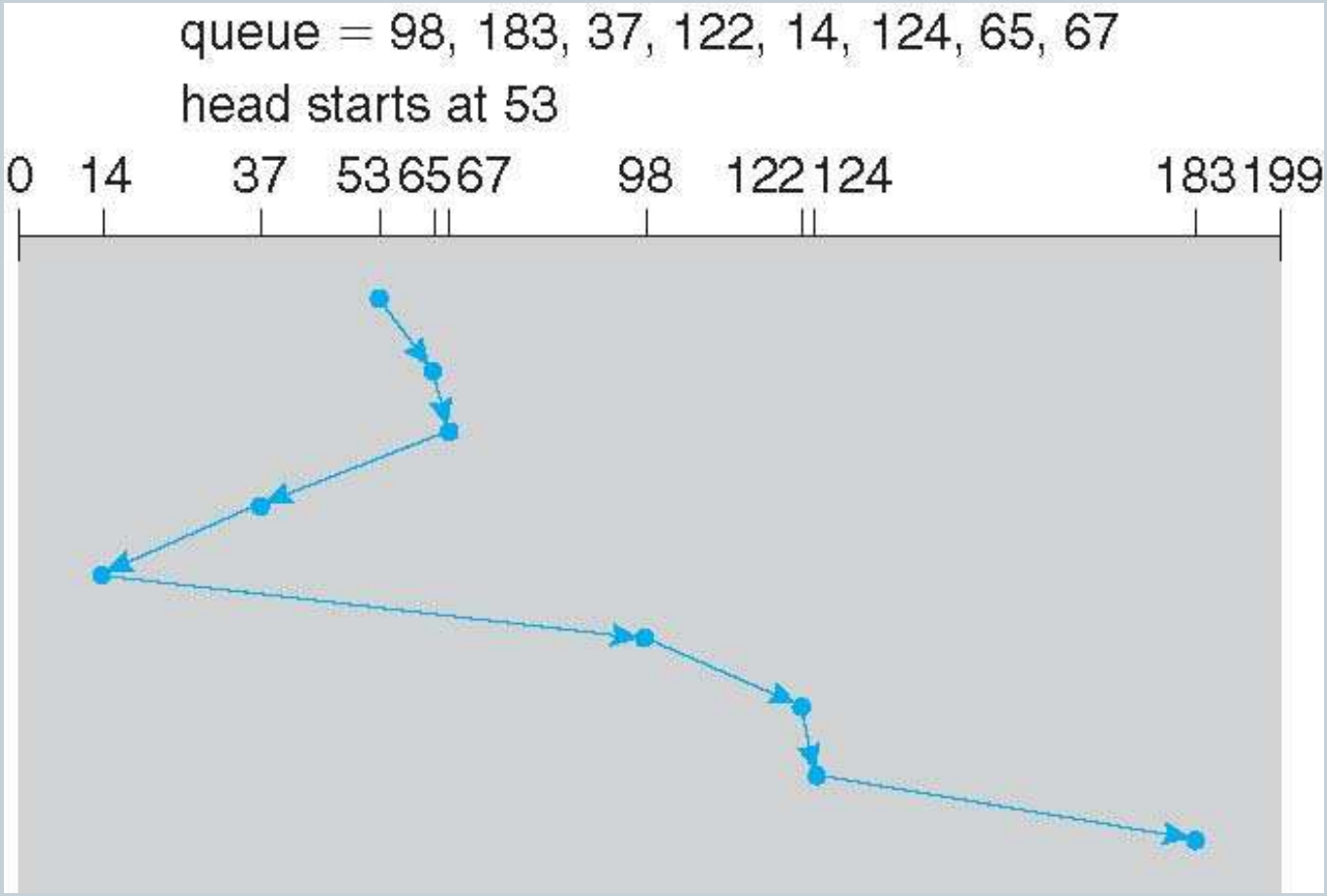


Shortest Seek Time First (SSTF)



- SSTF selects the request with the minimum seek time from the current head position
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests
- Illustration shows total head movement of 236 cylinders

Shortest Seek Time First (SSTF)



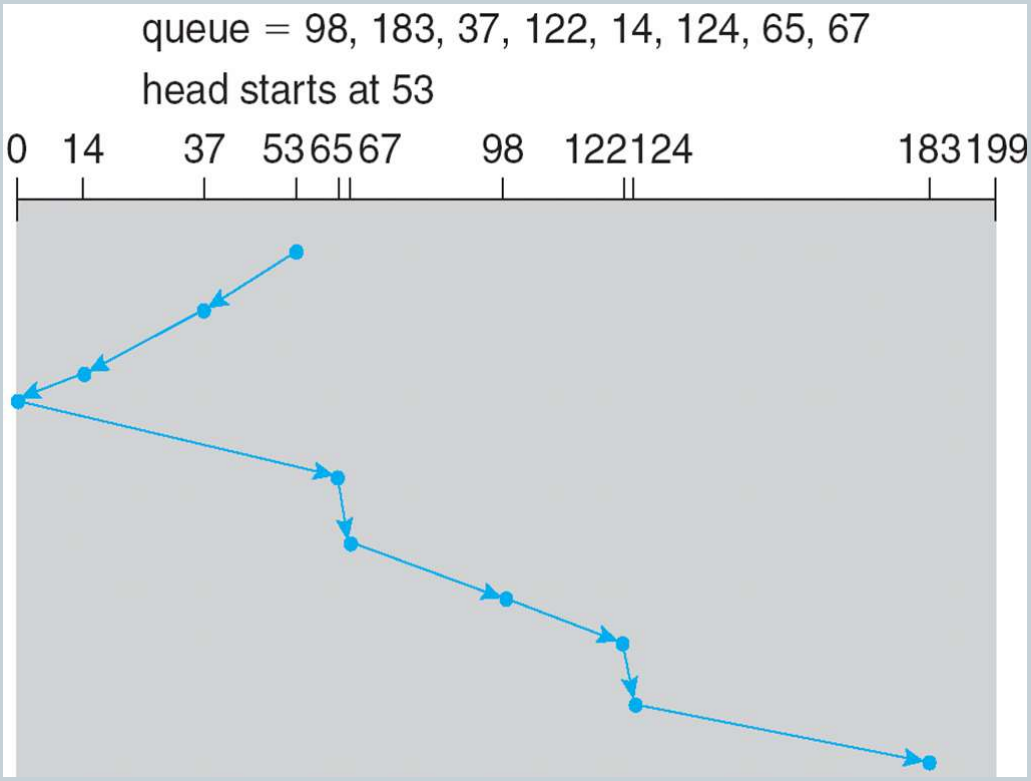
SCAN algorithm



- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- **SCAN algorithm** Sometimes called the **elevator algorithm**
- if requests are uniformly dense, largest density at other end of disk and those wait the longest

SCAN (Cont.)

- Illustration shows total head movement of 208 cylinders

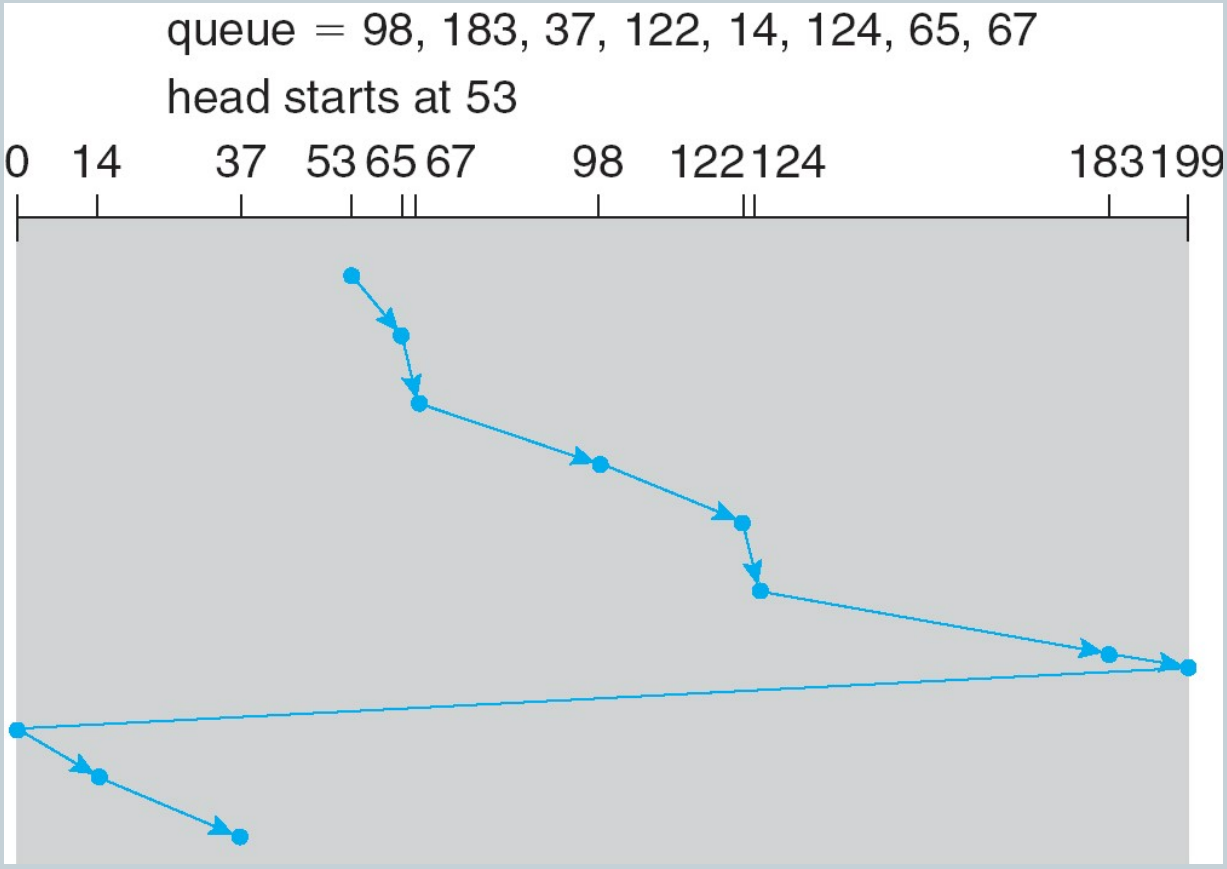


C-SCAN



- Provides a more uniform wait time than SCAN
- The head moves from one end of the disk to the other, servicing requests as it goes
  - When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one

C-SCAN (Cont.)

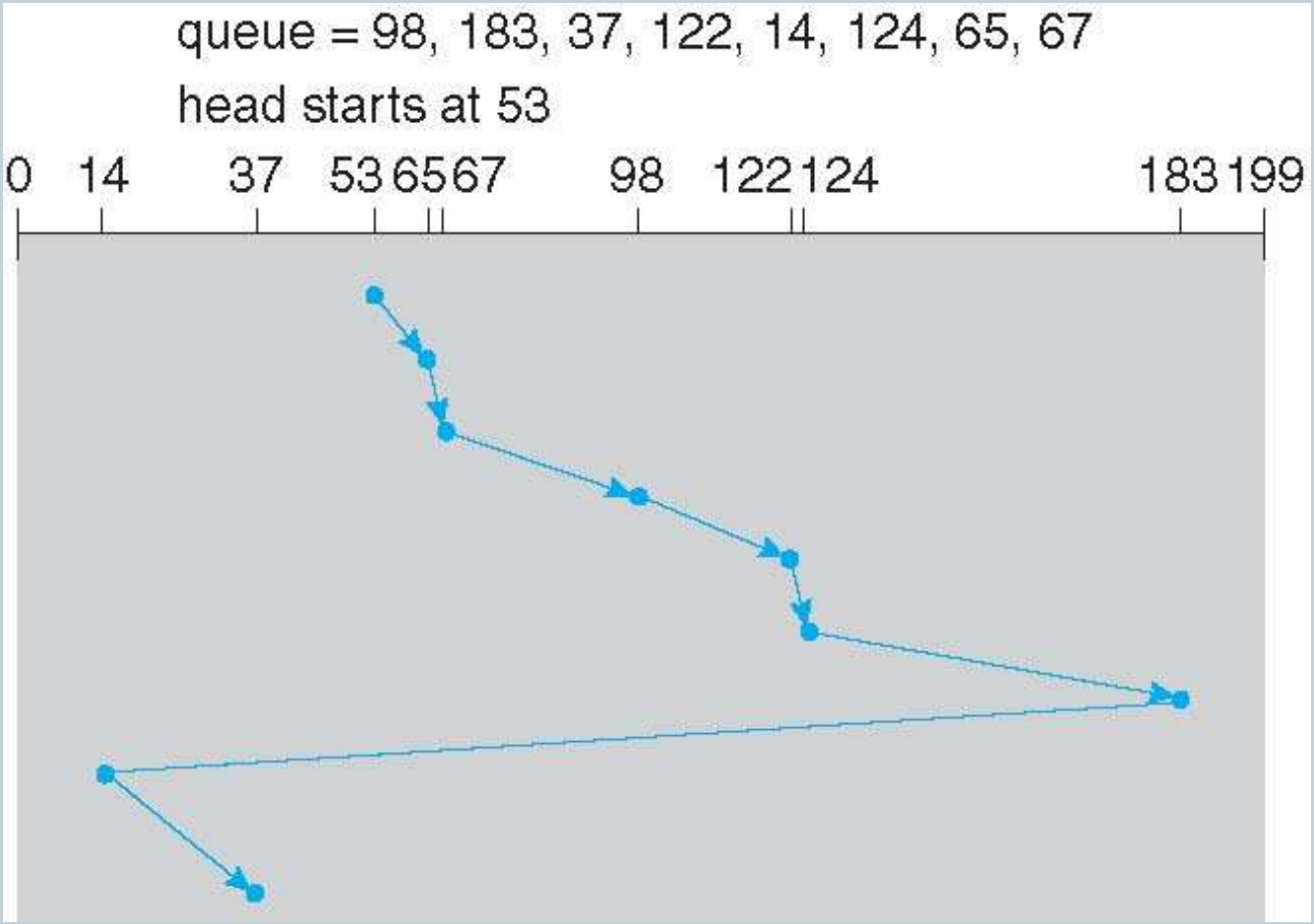

queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53
0
14
37
53
65
67
98
122
124
183
199

C-LOOK



- LOOK a version of SCAN, C-LOOK a version of C-SCAN
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk

C-LOOK (Cont.)


queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53
0
14
37
53
65
67
98
122
124
183
199

Selecting a Disk-Scheduling Algorithm



- Shortest Seek Time First (SSTF) is common and has a natural appeal
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
  - Less starvation
- Performance depends on the number and types of requests
- Requests for disk service can be influenced by the file-allocation method
  - And metadata layout

Selecting a Disk-Scheduling Algorithm



- The disk-scheduling algorithm should be written as a separate module of the operating system, allowing it to be replaced with a different algorithm if necessary
- Either SSTF or LOOK is a reasonable choice for the default algorithm

Disk Management

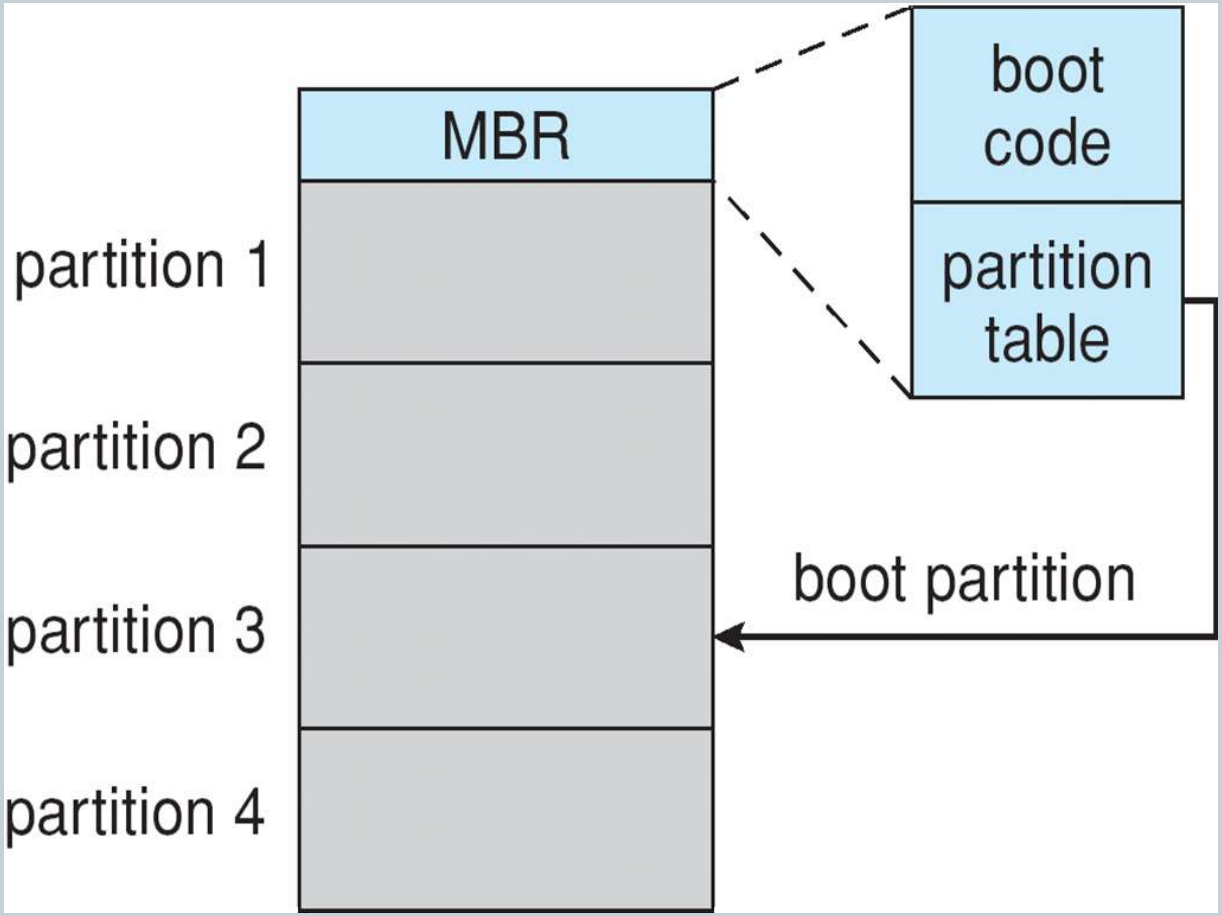
- **physical formatting** — Dividing a disk into sectors
  - Each sector can hold header information, plus data, plus error correction code (**ECC**)
  - Usually 512 bytes of data but can be selectable
- To use a disk to hold files, the operating system still needs to record its own data structures on the disk
  - **Partition** - one or more groups of cylinders, each treated as a logical disk
  - **Logical formatting** or "making a file system"
  - To increase efficiency most file systems group blocks into **clusters**
    - Disk I/O done in blocks
    - File I/O done in clusters

Disk Management (Cont.)



- Raw disk access for apps that want to do their own block management, keep OS out of the way (databases for example)
- Boot block initializes system
  - The bootstrap is stored in ROM
  - **Bootstrap loader** program stored in boot blocks of boot partition
- Methods such as **sector sparing** used to handle bad blocks

Booting from a Disk in Windows



Swap-Space Management



- Swap-space — Virtual memory uses disk space as an extension of main memory
  - Less common now due to memory capacity increases
- Swap-space can be carved out of the normal file system, or, more commonly, it can be in a separate disk partition (raw)

Stable-Storage Implementation



- Write-ahead log scheme requires stable storage
- Stable storage means data is never lost (due to failure, etc)
- To implement stable storage:
  - Replicate information on more than one nonvolatile storage media with independent failure modes
  - Update information in a controlled manner to ensure that we can recover the stable data after any failure during data transfer or recovery

Stable-Storage Implementation (Cont.)



- Disk write has 1 of 3 outcomes
  1. **Successful completion -** The data were written correctly on disk
  2. **Partial failure -** A failure occurred in the midst of transfer, so only some of the sectors were written with the new data, and the sector being written during the failure may have been corrupted
  3. **Total failure -** The failure occurred before the disk write started, so the previous data values on the disk remain intact

Stable-Storage Implementation (Cont.)



- If failure occurs during block write, recovery procedure restores block to consistent state
  - System maintains 2 physical blocks per logical block and does the following:
    1. Write to 1st physical
    2. When successful, write to 2nd physical
    3. Declare complete only after second write completes successfully